

Task 8: Integration and Demonstration (10 marks)

Team members: Chitambala Sikazwe (u25173384)
 , Gary Mull (u25071247), Louwrens Johansen (u25607414)

8.1 A coherent event simulation

main.cpp now builds and runs a single continuous simulation, runEventFlowDemo() in Task8Demo.cpp, rather than a set of separate per-task tests standing in for one. It reuses the same classes as every earlier task and introduces no new ones. Each requirement from the practical brief is covered and printed to the console as a clearly labelled step, so a marker can follow along without reading the code.

Requirement (Task 8.1)	Where it happens
Construction of the Composite	STEP 1: a root Venue, two Zones, one nested Precinct, five leaf types.
Observer registration	STEP 2: every zone and leaf attach()ed, kept separate from Composite ownership.
A Composite traversal/query	STEP 3: reportStatus() and getCapacity() recurse through the whole tree before anything happens.
At least three different notices	STEPS 4, 6, 8, 9: SCHEDULE_CHANGE, CAPACITY_ALERT, WEATHER_ALERT, EVACUATE and OPEN (five in total).
At least one cascading notification	Every issueNotice() call cascades; STEP 6 specifically reaches three runtime levels (root, River Side Zone, Food Court Precinct, Taco Stand).
A registration change	STEP 5: North Gate is detach()ed, a notice is sent and confirmed not to reach it, then it is re-attach()ed and reacts normally again.
A runtime reorganisation	STEP 7: transferChild() moves the Taco Stand from Food Court Precinct to Main Stage Zone, ownership and observer registration both move.
Clean shutdown	STEP 10: a single delete on the root, verified leak-free and dangling-pointer-free under AddressSanitizer/UndefinedBehaviorSanitizer.

Running ./eventflow on its own runs only this coherent demonstration. Running ./eventflow --regression first replays the individual Task 2, Task 3 and Task 4 demonstrations (unchanged) before the Task 8 demonstration, for a marker who wants to see one task's mechanism in isolation rather than the integrated story.

8.2 Demonstration route (about five minutes)

Run ./eventflow. The ten printed steps map directly onto a five minute walkthrough:

- STEP 1 to 3 (about 1 minute): point out the three levels of nesting and that `reportStatus()/getCapacity()` are genuinely recursive, not hardcoded per zone.
- STEP 4 (30 seconds): the first notice, show it reaching every leaf even though most have no reaction to it, that is itself a deliberate difference between leaf types.
- STEP 5 (1 minute): the registration change. Point out the gate does not react while detached, then does again once re-attached, this is the practical's non-negotiable rule 9 (attach, detach and notify all happening at runtime) satisfied directly.
- STEP 6 (1 minute): the weather alert. Walk through the five different leaf reactions and point out the notice reaching the Taco Stand three runtime levels below the root.
- STEP 7 (30 seconds): the reorganisation. Show the Taco Stand's ownership and its observer registration both moved by pointing at the status report before and after.
- STEP 8 to 9 (1 minute): evacuate, then recover. Any team member can be asked why a given leaf did or did not react and answer directly from that leaf's `update()` override.
- STEP 10 (a few seconds): clean shutdown, one delete call, no leak or dangling pointer left behind.

The Makefile

No task explicitly owned the Makefile the brief requires (non-negotiable rule 2: build with make, producing an executable named eventflow), so it was added here as part of integration. It compiles `main.cpp`, `Task2Testing.cpp`, `Task3Testing.cpp`, `Task4Testing.cpp`, `Task8Demo.cpp`, `Subject.cpp`, `CascadingZone.cpp` and `EventComposite.cpp`, and links them into eventflow.

`EventComponent.cpp` is deliberately not compiled as its own translation unit. It defines the five concrete leaf classes and every test/demo file that needs them includes it directly with `#include "EventComponent.cpp"`, so it is compiled once per file that includes it rather than separately. This was already how `Task2Testing.cpp` and `Task4Testing.cpp` used it, the Makefile just follows the same pattern for `Task8Demo.cpp` rather than changing it.

```
make      # builds eventflow
make clean # removes build artefacts
make docs  # regenerates docs/html via Doxygen
```

A `.gitignore` was also added, covering build artefacts (`*.o`, `eventflow`, `eventflow_asan`) and the stray precompiled main binary that was already sitting in the repository.

A note for the team on `CascadingZone::update()`

While building this demonstration, a small redundancy in Task 3's `CascadingZone::update()` became visible. On a `CLOSE` or `EVACUATE` notice, `update()` calls `this->close()` (which recurses through every child of the zone via the ordinary `Composite close()`, regardless of whether that child is registered as an observer) and then also calls `Subject::notify(notice)` (which delivers `update()` to every registered observer, and each leaf's own `update()` typically calls `close()` again as part of its reaction). The result, visible in STEP 8 of the demonstration, is that every leaf under a zone gets `close()`d twice for the same `EVACUATE` notice: once generically through the `Composite close()` and once again through its own `update()`. This is harmless, closing twice leaves a leaf in the same state as closing once, and every sanitizer check passed cleanly, but it is worth the team being aware of before Task 5's sequence diagrams are drawn, since a diagram of this interaction should show both calls if it is meant to

match the code exactly, and it may be simpler for whoever owns CascadingZone.cpp to have update() rely on one path or the other rather than both.